MODEL CONTEXT PROTOCOL INTERVIEW GUIDE

MCP Interview Preparation for Agentic AI Engineers

2026 Edition

Master Model Context Protocol, Tool-Calling Architecture, Secure AI Integrations, and Production-Grade Agent Systems for Technical Interviews

A focused technical interview handbook for engineers building secure, tool-using AI systems

MCP Agentic AI

The Definitive Technical Interview Guide

Published by AI Engineering Insider
aiengineeringinsider.com

First edition, 2025.
Version 1.0

Preface

The Model Context Protocol (MCP) is rewriting how AI systems interact with the world. In less than two years, MCP has grown from a research sketch into the *de facto* standard that ships inside Claude, Cursor, VS Code Copilot, and dozens of enterprise tools. The engineers who understand MCP deeply — who can design a secure, scalable, observable MCP deployment from scratch — are in extraordinary demand.

This book is written for them.

Who should read this book. Whether you are preparing for a senior-engineer interview at an AI-first company, a staff-level system-design round, or simply want to move from "I've used MCP" to "I can build and operate MCP at scale," this guide takes you through every layer of the protocol. We cover the foundational JSON-RPC 2.0 handshake, both canonical transports (stdio and Streamable HTTP), all six primitives (Tools, Resources, Prompts, Sampling, Elicitation, Roots), OAuth 2.1 authorization, the full security attack surface, production engineering (OpenTelemetry, HPA, circuit breakers, rate limiting), MCP Apps with iframe UIs, and agentic multi-agent patterns.

How the book is structured. Sixteen chapters, organized into four parts:

- **Part I (Chapters 1–4):** Foundations, architecture, client setup, and the protocol lifecycle.
- **Part II (Chapters 5–8):** Transports, building servers, advanced client patterns, and OAuth authorization.
- **Part III (Chapters 9–12):** Security hardening, advanced server techniques, testing & CI, and production engineering.
- **Part IV (Chapters 13–16):** MCP Apps & UI extensions, agentic framework integrations, evaluation playbooks, and a comprehensive interview-prep reference chapter.

Each chapter opens with a **Chapter Overview** box, includes **diagrams** to make architectural concepts visual, and closes with a **Mock Interview** section of at least ten questions tagged by category: Theory, Syntax, and Production.

A note on versions. The content reflects MCP Specification 2025-03-26 and the FastMCP 2.x Python SDK. Where the specification has evolved rapidly, we note the version delta explicitly.

AI Engineering Insider. This book is published by AI Engineering Insider, a technical resource for professionals building on top of foundation models and agentic systems. For errata, supplementary materials, and new chapters as the protocol evolves, visit `aiengineeringinsider.com`.

Good luck in your interviews — and beyond.

The AI Engineering Insider Team

Contents

Preface **v**

1 MCP Foundations **1**
1.1 What is MCP? . . . 1
1.2 MCP vs Everything Else . . . 2
1.3 MCP Use Cases . . . 3
Mock Interview Questions . . . 4
Q1.1 [Theory] What problem does MCP solve that raw function calling does not? 4
Q1.2 [Theory] Which RPC specification underpins MCP, and why was it chosen over alternatives like gRPC or GraphQL? . . . 4
Q1.3 [Theory] Name the four primitive capability categories MCP defines and give one concrete example of each. . . . 5
Q1.4 [Syntax] Write the minimal valid JSON-RPC 2.0 request to call an MCP tool named `get_weather` with argument `city="Tokyo"`. . . . 5
Q1.5 [Syntax] What is the correct JSON-RPC error response when a method is not found? . . . 5
Q1.6 [Theory] How does MCP relate to the A2A (Agent-to-Agent) protocol? . . . 5
Q1.7 [Theory] Give two reasons why an enterprise might prefer MCP over embedding tool logic directly in a LangChain agent. . . . 5
Q1.8 [Production] A team hardcoded `"protocolVersion":"2024-11-05"` in their MCP server. Describe the failure mode and the fix. . . . 6
Q1.9 [Production] A customer support MCP server exposes an `order_lookup` tool. A bug causes it to return orders from other customers. What MCP-level controls should have prevented this? . . . 6
Q1.10 [Production] Your MCP server is deployed as a container in Kubernetes. Latency spikes to 10 s on every first request after a pod restart. What is the most likely cause and fix? . . . 6
Q1.11 [Theory] What distinguishes an MCP *notification* from an MCP *request*? . . . 6

2 MCP Architecture **7**
2.1 Host, Client, Server Model . . . 7
2.2 JSON-RPC 2.0 Message Mechanics . . . 8
2.3 Capability Negotiation . . . 9
Mock Interview Questions . . . 11
Q2.1 [Theory] Explain the difference between a Host and an MCP Client. Why does the specification keep them separate? . . . 11
Q2.2 [Theory] Why must a Host create one Client per Server rather than one shared Client? . . . 11
Q2.3 [Syntax] Write the Python snippet (MCP SDK) that sends a `tools/list` request and prints each tool's name. . . . 11

Q2.4 [Syntax] What JSON-RPC error code should a server return if it receives a `tools/call` with an argument that fails schema validation? . . . 11
Q2.5 [Theory] What is the purpose of the `notifications/initialized` message? What happens if it is omitted? . . . 11
Q2.6 [Theory] How does MCP handle the case where the client and server support different protocol versions? . . . 12
Q2.7 [Syntax] Show the structure of a JSON-RPC notification (not a request). Name two MCP notifications that use this shape. . . . 12
Q2.8 [Theory] A server advertises `"sampling":{}` in its `InitializeResult`. What does this mean? . . . 12
Q2.9 [Production] During a load test your team discovers that MCP request IDs collide when two threads issue requests concurrently. What is the correct fix? . . . 12
Q2.10 [Production] Your MCP server is connected to a stateful database. A client disconnects unexpectedly mid-session without sending a shutdown message. How should the server clean up? . . . 13
Q2.11 [Theory] Can a server call `resources/read` on the client? Explain. . . . 13

3 Connecting to MCP: Client Setup **14**
3.1 MCP Client SDK Setup . . . 14
3.2 Context Injection via MCP Resources . . . 15
Mock Interview Questions . . . 17
Q3.1 [Syntax] Write the TypeScript code to connect an MCP client over stdio and list all available resources. . . . 17
Q3.2 [Theory] What is the difference between a resource *template* and a static resource in the MCP spec? . . . 17
Q3.3 [Theory] What async library does the Python MCP SDK use, and why does this matter for production deployment? . . . 18
Q3.4 [Syntax] How do you handle a `tool call` response that indicates an error in the Python SDK? . . . 18
Q3.5 [Theory] What does `session.initialize()` do under the hood? . . . 18
Q3.6 [Production] Your MCP client in production intermittently fails with "server process exited unexpectedly." How do you diagnose and fix? . . . 18
Q3.7 [Theory] Explain resource subscriptions and give a use-case where they are preferable to polling. . . . 18
Q3.8 [Syntax] Show the JSON for a `resources/read` request for the URI `db://orders/42`. . . . 19
Q3.9 [Production] A model's context window is exhausted by a large resource. List three mitigation strategies using MCP primitives. . . . 19
Q3.10 [Theory] What Python version is required by the MCP SDK and what async paradigm does it use? . . . 19

4 Lifecycle & Protocol Primitives **20**
4.1 MCP Connection Lifecycle . . . 20
4.2 Progress Notifications & Logging . . . 21
4.3 Cancellation . . . 22
4.4 Pagination . . . 23
Mock Interview Questions . . . 24

Q4.1 [Theory] Describe the three phases of an MCP session and what happens if the server receives a `tools/call` before initialization is complete. . . . 24
Q4.2 [Syntax] Write Python code to call a tool with a progress token and print each progress update as it arrives. . . . 24
Q4.3 [Theory] Why does MCP use cursor-based pagination instead of offset-based pagination? . . . 25
Q4.4 [Syntax] What JSON does the client send to cancel request ID 42? . . . 25
Q4.5 [Theory] If a client sends a cancel notification for request 42, can the server still send the result for request 42? . . . 25
Q4.6 [Production] A production MCP server processes long database exports. Describe how you would implement graceful cancellation to avoid leaving partial exports on disk. . . . 26
Q4.7 [Syntax] Show the `tools/list` request that fetches the second page using cursor `"c2_xyz"`. . . . 26
Q4.8 [Theory] What does it mean for a cursor to be *opaque*? Give two things the client must *not* do with a cursor. . . . 26
Q4.9 [Production] Your pagination loop runs successfully in staging (100 tools) but hangs indefinitely in production (50,000 tools across 10 servers). Identify two failure modes and fixes. . . . 26
Q4.10 [Theory] What is the difference between `notifications/progress` and `notifications/message` (logging)? . . . 26
Q4.11 [Production] After a version upgrade, some clients on protocol version `2024-11-05` connect to your new server (version `2025-03-26`). What degraded behaviour should you document for those clients? . . . 27

Part 1 Quick-Reference **28**

5 MCP Transports **29**
5.1 stdio Transport . . . 29
5.2 Streamable HTTP Transport . . . 30
5.3 Transport Security . . . 31
Mock Interview Questions . . . 33
Q5.1 [Theory] What are the two canonical MCP transports? In one sentence each, give the primary use case for each. . . . 33
Q5.2 [Theory] Why does the Streamable HTTP transport use *both* POST and GET to a single endpoint? . . . 33
Q5.3 [Syntax] Show the HTTP request headers a client must include when calling `tools/call` on an existing Streamable HTTP session. . . . 33
Q5.4 [Theory] Explain the DNS rebinding attack against a local MCP HTTP server and the two spec-mandated mitigations. . . . 33
Q5.5 [Syntax] Write a Python `aiohttp` middleware snippet that validates the `Origin` header for an MCP HTTP server. . . . 34
Q5.6 [Production] After deploying an MCP server, you notice that some SSE streams to clients disconnect every 30 s and the client never reconnects. Diagnose the issue. . . . 34
Q5.7 [Theory] What framing does the stdio transport use to delimit messages, and what constraint does this place on message content? . . . 34

Q5.8 [Production] A production MCP stdio server occasionally produces zombie processes. What causes this and how do you fix it? . . . 34
Q5.9 [Theory] What is the purpose of the `Last-Event-ID` SSE header in the Streamable HTTP transport? . . . 34
Q5.10 [Production] Your remote MCP server is exposed without authentication because "it only does read-only lookups." What attack vectors remain open? . . . 35
Q5.11 [Syntax] What HTTP status code should an MCP server return when a client sends a request with an unrecognized `MCP-Session-Id`? . . . 35

6 Building Servers: Tools, Prompts & Resources 36
6.1 Tool Definition & Design . . . 36
6.2 Tool Discovery & Execution . . . 37
6.3 Tool Chaining Patterns . . . 39
6.4 Streaming Tool Results . . . 40
6.5 Resource Definition & Access . . . 40
6.6 Prompts . . . 41
6.7 Tool & Resource Security . . . 42
Mock Interview Questions . . . 43
Q6.1 [Theory] Why is the `description` field in a tool definition the most important field for production reliability? . . . 43
Q6.2 [Syntax] Write the Python SDK decorator that registers a tool named `list_users` that accepts a `role` string filter and returns a list. . . . 43
Q6.3 [Theory] What is the difference between returning `isError: true` vs raising an exception in a tool handler? . . . 43
Q6.4 [Syntax] Show how to implement parallel fan-out — calling three tools simultaneously and aggregating their results in Python. . . . 43
Q6.5 [Theory] Explain the prompt injection risk from tool results and describe two MCP-level mitigations. . . . 44
Q6.6 [Syntax] Demonstrate path traversal prevention for a `read_file` tool that restricts access to `/workspace/`. . . . 44
Q6.7 [Production] A tool that calls an external API is registered with no timeout. Describe the production failure mode and the fix. . . . 44
Q6.8 [Theory] What is the difference between a static resource and a resource template? Give an example of when you would choose each. . . . 44
Q6.9 [Syntax] Write the prompts/get request JSON to retrieve a prompt named `summarise` with argument `text="Hello world"`. . . . 45
Q6.10 [Production] Your MCP server is receiving `tools/call` requests before `tools/list` has been called by the client. The client hardcoded the tool name. Is this valid, and what can go wrong? . . . 45
Q6.11 [Theory] What MCP field allows a server to return machine- parseable structured output alongside human-readable text, and why does this matter? . . . 45

7 MCP Clients: Advanced Use & Best Practices 46
7.1 Roots . . . 46
7.2 Sampling . . . 47
7.3 Elicitation . . . 48

7.4 Advanced Client Patterns & Best Practices 49
Mock Interview Questions 50
Q7.1 [Theory] What is the Roots capability and why does it exist? 50
Q7.2 [Theory] Explain the direction reversal in the Sampling primitive. Who initiates the request and why? 50
Q7.3 [Syntax] Write the JSON for a `sampling/createMessage` request that asks the LLM to translate text to French, preferring a fast model. 50
Q7.4 [Theory] What three `action` values can a client return in response to an `elicitation/create` request, and what does each mean? 50
Q7.5 [Syntax] Write Python code that checks whether the connected server supports resource subscriptions before attempting to subscribe. 51
Q7.6 [Production] Your multi-server agent crashes all sessions when one MCP server times out. Diagnose and fix. 51
Q7.7 [Theory] What does `includeContext: "thisServer"` do in a sampling request? 51
Q7.8 [Production] A server sends `elicitation/create` to collect a database password. Is this valid? What should the client do? 51
Q7.9 [Theory] What is capability probing and why should clients always perform it before calling optional MCP methods? 52
Q7.10 [Production] Describe how you would implement LRU caching for `tools/list` in a long-running Python MCP client. 52
Q7.11 [Theory] How does the Roots capability protect against a rogue MCP server attempting to exfiltrate `/etc/shadow`? 52

8 MCP Authorization **54**
8.1 OAuth 2.1 Foundations for MCP 54
8.2 MCP Auth Flows & Enterprise Patterns 55
Mock Interview Questions 57
Q8.1 [Theory] Which OAuth grant type does MCP recommend and why is the implicit grant explicitly forbidden? 57
Q8.2 [Theory] Explain PKCE step by step. Why does it prevent authorization code interception? 57
Q8.3 [Syntax] Write the Python snippet to compute a PKCE `code_challenge` from a `code_verifier`. 58
Q8.4 [Theory] What is the OAuth discovery document and how does an MCP client use it? 58
Q8.5 [Syntax] Show the HTTP request an MCP Host sends to obtain an access token after receiving the authorization code, including the PKCE verifier. 58
Q8.6 [Theory] What is the recommended access token lifetime for a remote MCP server and why? 59
Q8.7 [Production] An MCP server receives a valid JWT but the token's `scope` claim does not include `orders:write`. The server must execute a refund tool. What should it do? 59
Q8.8 [Production] Where should an MCP desktop host store OAuth refresh tokens? Give two wrong answers commonly seen in the wild. 59
Q8.9 [Theory] How does IdP federation work in an enterprise MCP deployment? 59

Q8.10 [Production] Dynamic client registration is enabled on your MCP authorization server. An attacker registers a malicious client with a legitimate-looking `client_name`. What controls prevent abuse? . . . 59
Q8.11 [Theory] What is the difference between an access token, a refresh token, and an ID token in the MCP context? . . . 60

Part 2 Quick-Reference **61**

9 MCP Security **62**
9.1 MCP Attack Surface . . . 62
9.2 Security Hardening & Anti-Patterns . . . 65
Mock Interview Questions . . . 66
Q9.1 [Theory] Explain the confused deputy problem in the MCP context. Give a concrete example and the correct fix. . . . 66
Q9.2 [Theory] Describe an SSRF attack via an MCP `fetch_url` tool. Which IP ranges must be blocked? . . . 66
Q9.3 [Syntax] Write Python code to validate that a URL is not pointing to a private IP range before fetching it. . . . 66
Q9.4 [Theory] What is tool poisoning and how does it differ from prompt injection? 67
Q9.5 [Production] Your audit log shows a single OAuth token making 10,000 `tools/call` requests in 60 seconds. What controls should have triggered and what is the immediate remediation? . . . 67
Q9.6 [Theory] Why must session IDs be cryptographically random UUIDs rather than sequential integers? . . . 67
Q9.7 [Syntax] Show the Python code to hash tool call arguments before writing to an audit log, to prevent PII leakage. . . . 67
Q9.8 [Production] An MCP server is deployed without a tool-call timeout. A downstream service hangs. Describe the cascading failure. . . . 68
Q9.9 [Theory] What is the token passthrough anti-pattern and why is it dangerous? 68
Q9.10 [Theory] List three controls that mitigate prompt injection from tool results. 68
Q9.11 [Production] During a pentest your team discovers that the MCP server logs the full JSON body of every `tools/call` request including customer credit card numbers passed in arguments. Describe your immediate and long-term fixes. . . . 68

10 Building Servers: Advanced Use & Utilities **69**
10.1 SDK Utilities Deep Dive . . . 69
10.2 Integrating Client Capabilities into Servers . . . 70
10.3 Multi-Server Composition . . . 71
10.4 Publishing & Sharing Your MCP Server . . . 72
Mock Interview Questions . . . 73
Q10.1 [Theory] What is the FastMCP layer in the Python SDK and why would you use it over the low-level Server API? . . . 73
Q10.2 [Syntax] Show how a server tool can request an LLM sampling completion through the MCP client using the Python SDK context. . . . 73
Q10.3 [Theory] Why might you build an aggregator MCP server instead of having the host connect to multiple servers directly? . . . 74

Q10.4 [Syntax] Write the namespace prefixing logic for an aggregator that serves tools from two upstream servers named `crm` and `billing`. 74
Q10.5 [Production] Your aggregator server routes tool calls to three upstreams. One upstream goes down. What should the aggregator do? 74
Q10.6 [Theory] What fields should a production `mcp.json` manifest include? . 74
Q10.7 [Production] A server tool accumulates memory because it builds a large list in RAM before returning. How would you fix this using SDK streaming patterns? 75
Q10.8 [Syntax] Write the FastMCP prompt registration for a `code_review` prompt that accepts `diff` and `language` arguments. 75
Q10.9 [Theory] How does capability merging work in an aggregator server, and what happens when two upstreams have conflicting capability declarations? 75
Q10.10 [Production] You need to publish an MCP server that works both as a local stdio process and as a remote HTTP service. How do you structure the entry point? 75
Q10.11 [Theory] What does `ctx.list_roots()` return and how should a server use it to enforce access boundaries? 76

11 Testing, Debugging & CI/CD **77**
11.1 Testing MCP Servers 77
11.2 MCP Inspector & Debugging 79
11.3 CI/CD, Dockerising & Schema Versioning 80
Mock Interview Questions 81
Q11.1 [Theory] Describe the three levels of the MCP test pyramid and what each tests. 81
Q11.2 [Syntax] Write a pytest fixture that creates an in-process MCP client session connected to a test server. 81
Q11.3 [Theory] What does the MCP Inspector tool do and how is it launched against a stdio server? 82
Q11.4 [Production] Your CI schema regression test fails after a team member added a new required field `tenant_id` to an existing tool's `inputSchema`. Is this a breaking change? What is the correct migration path? 82
Q11.5 [Syntax] Show a Dockerfile command that runs the MCP server as a non-root user. 82
Q11.6 [Theory] What is the difference between a breaking and a non-breaking schema change? Give one example of each. 83
Q11.7 [Production] The MCP Inspector shows `-32601 Method not found` when calling `create_Issue`. What are the two most common root causes? 83
Q11.8 [Theory] Why should integration tests use the in-process memory transport rather than spawning a real subprocess? 83
Q11.9 [Production] Your team wants to validate tool schemas automatically in the CI pipeline before any code is merged. Describe the implementation. 83
Q11.10 [Theory] What Python SDK function provides an in-process transport for testing, and what does it return? 83
Q11.11 [Production] After a deployment, clients start receiving `-32602 Invalid params` on a tool call that worked before. The tool's inputSchema was not intentionally changed. How do you diagnose? . . 84

12 MCP Production Engineering **85**
12.1 MCP-Specific Observability . . . 85
12.2 Scalability & Deployment . . . 86
12.3 Cost Control & Governance . . . 88
Mock Interview Questions . . . 89
Q12.1 [Theory] What are the three observability pillars and how does each apply specifically to an MCP server? . . . 89
Q12.2 [Syntax] Show the key OpenTelemetry attributes to set on a tool-call span. 89
Q12.3 [Theory] Why should MCP rate limits be applied per OAuth token subject rather than per IP address? . . . 89
Q12.4 [Production] Your MCP server is on Kubernetes and P99 tool call latency spikes from 50 ms to 8 s under load. Walk through your diagnosis. . . . 90
Q12.5 [Theory] What is the governance gateway pattern and when is it required? 90
Q12.6 [Syntax] Write a Kubernetes liveness probe configuration for an MCP HTTP server that checks the server's health endpoint. . . . 90
Q12.7 [Production] An enterprise customer reports that a single rogue agent consumed $12,000 of API credits in one hour via MCP sampling calls. What controls would have prevented this? . . . 90
Q12.8 [Theory] What is the difference between a readiness probe and a liveness probe in the context of an MCP Kubernetes deployment? . . . 91
Q12.9 [Production] Describe how you would implement multi-tenant cost attribution for an MCP server serving 500 enterprise customers. . . . 91
Q12.10 [Theory] What is a circuit breaker in the MCP context and how does it protect against cascading failures? . . . 91
Q12.11 [Production] Your on-call alert fires: `mcp.tool. error_rate > 5%` for 3 minutes. Walk through your runbook. . . . 91

Part 3 Quick-Reference **93**

13 MCP Apps & UI Extensions **94**
13.1 MCP Apps Architecture . . . 94
13.2 MCP App Patterns . . . 96
Mock Interview Questions . . . 97
Q13.1 [Theory] What is a `ui://` resource and how does it differ from a standard `file://` resource? . . . 97
Q13.2 [Syntax] Write the FastMCP Python code to register a `ui://widget/counter` resource that returns a minimal HTML counter app. . . . 97
Q13.3 [Theory] Why must the Host NOT combine `allow-scripts` and `allow-same-origin` in the MCP App iframe sandbox? . . . 97
Q13.4 [Theory] How does a MCP App UI receive real-time updates from the server without polling? . . . 97
Q13.5 [Production] A MCP App dashboard goes blank after 30 seconds. The browser console shows `postMessage origin mismatch`. Diagnose and fix. . . . 98
Q13.6 [Theory] What is tool-to-UI linking and how does the Host know to open an iframe when a tool returns? . . . 98
Q13.7 [Syntax] Show the JavaScript to call a tool and display the result inside an MCP App using the bridge API. . . . 98

Q13.8 [Production] Your MCP App config form calls a `save_config` tool that triggers a server restart. Users report the form freezes after submission. Fix the UX. . . . 98
Q13.9 [Theory] What Content Security Policy headers should an MCP App page include to prevent XSS? . . . 99
Q13.10 [Theory] Describe two MCP App patterns suited to an enterprise HR system. . . . 99

14 MCP with Agentic AI Frameworks **100**
14.1 MCP + LangGraph / LangChain . . . 100
14.2 MCP + Claude (API & Desktop) . . . 101
14.3 MCP + OpenAI / Ollama . . . 102
14.4 Multi-Agent Systems with MCP . . . 103
14.5 MCP + Java Frameworks . . . 104
Mock Interview Questions . . . 105
Q14.1 [Theory] How does the LangChain MCP adapter convert MCP tools to LangChain `StructuredTool` objects? . . . 105
Q14.2 [Syntax] Show the minimal Claude Desktop config to add a remote HTTP MCP server with an authorization header. . . . 105
Q14.3 [Theory] What is the Planner/Executor/Verifier pattern and why does separating OAuth scopes across agents matter? . . . 105
Q14.4 [Syntax] Write the Python code to convert an MCP tool list to Anthropic API tool definitions. . . . 105
Q14.5 [Production] A LangGraph agent enters an infinite retry loop when an MCP tool returns `isError:true`. How do you fix this? . . . 106
Q14.6 [Theory] How does Ollama's tool call format differ from the MCP protocol, and what is the adapter's job? . . . 106
Q14.7 [Theory] What Spring AI annotation registers a Java method as an MCP tool, and what fields does it expose? . . . 106
Q14.8 [Production] Your multi-agent system has a Planner, Executor, and Verifier all connected to the same MCP server. The Verifier incorrectly calls a write tool. What governance control prevents this? . . . 107
Q14.9 [Theory] What is the `includeContext` parameter in `sampling/createMessage` and how does it affect multi-agent systems? . . . 107
Q14.10 [Syntax] Show how to register a tool in LangChain4j using the MCP client and invoke it. . . . 107
Q14.11 [Production] Two agents share the same `ClientSession` object. Describe the race condition and the correct fix. . . . 107

15 MCP Agent Evaluation & Use Case Playbooks **108**
15.1 Evaluating MCP Agents . . . 108
15.2 Customer Support Agent Playbook . . . 109
15.3 AI Coding Assistant Playbook . . . 110
15.4 Enterprise Workflow Agent Playbook . . . 111
Mock Interview Questions . . . 112
Q15.1 [Theory] Name the six dimensions of MCP agent evaluation and explain why tool selection accuracy is the most fundamental. . . . 112

Q15.2 [Production] A customer support agent incorrectly refunds customers who are outside the refund policy window. Which evaluation dimension detects this and how would you fix it? . . . 112
Q15.3 [Theory] In the coding assistant playbook, why is `Roots` essential rather than optional? . . . 112
Q15.4 [Syntax] Write a golden-set test case entry for an `order_lookup` tool in the customer support agent evaluation. . . . 112
Q15.5 [Production] Your enterprise workflow agent is leaking SharePoint document titles in the Slack audit log. How do you fix this without losing traceability? . . . 112
Q15.6 [Theory] How would you measure the grounding metric for an MCP-powered customer support agent? . . . 113
Q15.7 [Production] The coding assistant's `run_tests` tool runs in the developer's local environment and takes 5 minutes. Users abandon the agent after 30 seconds. Fix the UX without reducing test coverage. . . . 113
Q15.8 [Theory] What is the role of the governance gateway in the enterprise workflow playbook and how does it interact with MCP elicitation? . . . 113
Q15.9 [Theory] Why is hallucination rate a distinct metric from grounding, and how do they interact? . . . 113
Q15.10 [Syntax] Write the Python evaluation loop that computes tool-selection accuracy over a golden set. . . . 114
Q15.11 [Production] After a Jira MCP server upgrade, tool-selection accuracy drops from 92% to 61%. What is the most likely cause? . . . 114

16 Interview Preparation & Reference **115**
16.1 MCP Conceptual Q&A — 40 Essential Questions . . . 115
16.2 System Design Interview Scenarios . . . 117
16.3 MCP Coding Challenges . . . 119
16.4 Cheatsheets . . . 121
16.5 MCP Glossary . . . 122

Complete Book Quick-Reference **126**

About AI Engineering Insider **127**

CHAPTER 1

MCP Foundations

Chapter Overview

MCP gives AI applications a protocol boundary for tools, context, and host-mediated capabilities. The starting point is deliberately practical: what MCP standardizes on the wire, how JSON-RPC 2.0 shapes every exchange, and where MCP is a better fit than raw function calling or an in-process framework adapter. The goal is a clean mental model you can use in architecture reviews and senior interview discussions.

1.1 What is MCP?

Model Context Protocol (MCP) is an open, vendor-neutral application-layer protocol introduced by Anthropic in November 2024 and rapidly adopted across the industry through 2025. Its central premise is elegantly simple: *separate the AI model from the tools and data it needs to act on the world*, and define a single, stable contract for how they communicate.

Before MCP, every AI product team wrote bespoke glue code — custom JSON schemas for tool definitions, proprietary streaming formats for partial results, ad-hoc authentication wrappers. The result was a combinatorial explosion: N models $\times$ M tool providers $= N \cdot M$ integrations. MCP collapses this to $N + M$ by introducing a common hub-and-spoke protocol.

Protocol Definition

At the wire level MCP is a **JSON-RPC 2.0** protocol transported over one of two channels: `stdio` (for local subprocesses) or **Streamable HTTP** (for remote servers). Every message is a UTF-8-encoded JSON object conforming to the JSON-RPC 2.0 specification:

```
{
  "jsonrpc": "2.0",
  "id":      1,
  "method":  "tools/call",
  "params":  {
    "name":      "search_docs",
    "arguments": { "query": "MCP transport security" }
  }
}
```

Listing 1.1: JSON-RPC 2.0 request skeleton used by MCP

MCP layers its own method namespace (`tools/`, `resources/`, `prompts/`, etc.) directly on top of JSON-RPC 2.0 without altering the base framing. Error codes follow the JSON-RPC convention (`-32700` parse error through `-32603` internal error) with MCP-specific codes starting at `-32000`.

The Protocol Stack

Figure 1.1 shows the layered architecture:

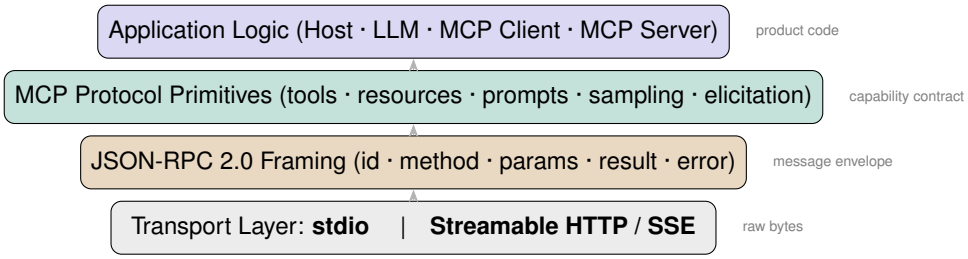

Figure 1.1: MCP protocol stack. Each layer adds semantic structure without replacing the layer beneath it.

Ecosystem Overview 2025–26

By early 2026 the MCP ecosystem had grown to include:

- **Official SDKs**: Python (`mcp`), TypeScript (`@modelcontextprotocol/sdk`), Java (Spring AI MCP, LangChain4j), Go, Rust, and Kotlin.
- **Host applications**: Claude Desktop, Cursor, VS Code Copilot, Continue, Zed, and dozens of enterprise forks.
- **Server registry**: `mcp.so` and the official Anthropic registry list thousands of community MCP servers.
- **Protocol version**: `2025-03-26` is the current stable version; clients and servers negotiate backwards compatibility during the initialization handshake.

Production Issue

Anti-pattern — hardcoding the protocol version string.
Teams that hardcode `"protocolVersion":"2024-11-05"` (the initial release) in their servers silently fail capability negotiation with 2025-era clients that no longer announce that version. Always derive the version constant from the SDK (`LATEST_PROTOCOL_VERSION`) and let the handshake select the best mutual version.

1.2 MCP vs Everything Else

Interviewers routinely ask candidates to justify an architectural choice. This section arms you with a precise decision table and the reasoning behind each cell.

The Comparison Landscape

Decision Table: When to Use MCP

A2A Protocol

The **Agent-to-Agent (A2A)** protocol (Google, 2025) complements MCP rather than competing with it. A2A governs how autonomous agents communicate with *each other* at the task/intent level; MCP governs how an agent communicates with *tools and data sources* at the capability level. A production multi-agent system commonly uses A2A for inter-agent coordination and MCP for each agent's tool access.

Dimension	MCP	OpenAI Function Calling	LangChain Tools
Scope	Cross-vendor protocol	Provider-specific extension	Python library abstraction
Transport	stdio / HTTP+SSE	HTTPS to OpenAI API	In-process function call
Server locality	Separate process/host	No separate process	Same process
Streaming results	Native (partial outputs)	Not in tool results	Via callbacks
Auth standard	OAuth 2.1 + PKCE	API key only	Library-specific
Multi-model	✓(any host)	× (OpenAI only)	✓
Human-in-the-loop	Native (elicitation)	Manual implementation	Manual
Schema versioning	Protocol-level	Provider-managed	None

Table 1.1: MCP vs competing tool-use paradigms.

1.3 MCP Use Cases

Understanding where MCP shines in production is essential for system design interviews.

IDE Agents

Cursor, VS Code with Copilot, and Continue all embed MCP clients. An MCP server exposes the repository's file tree as resources (`file://` URIs), compiles the project via a `build` tool, and runs tests through a `run_tests` tool. The IDE host injects resource content into the model's context window automatically, without copying files into the chat.

Customer Support Bots

A support agent uses an MCP server backed by the company's CRM. The server exposes an `order_lookup` tool (reads from Postgres), a `refund_policy` resource (a PDF extracted to markdown), and a `escalate_to_human` tool that publishes to a queue. OAuth 2.1 scopes ensure the agent can only read orders belonging to the authenticated customer.

Enterprise Assistants and Desktop Automation

Claude Desktop connects over stdio to locally-running MCP servers that can read the filesystem (with `roots` boundaries), run shell commands, and interact with macOS Accessibility APIs. Because the server runs in the same user session, no network egress is required and sensitive data never leaves the machine.

Research Agents

A research agent fans out across multiple MCP servers simultaneously — one for Semantic Scholar, one for arXiv, one for an internal knowledge base — aggregating results via the client's parallel tool call capability. Pagination cursors allow streaming thousands of results without loading them all into the context window.

Mock Interview Questions — Chapter 1

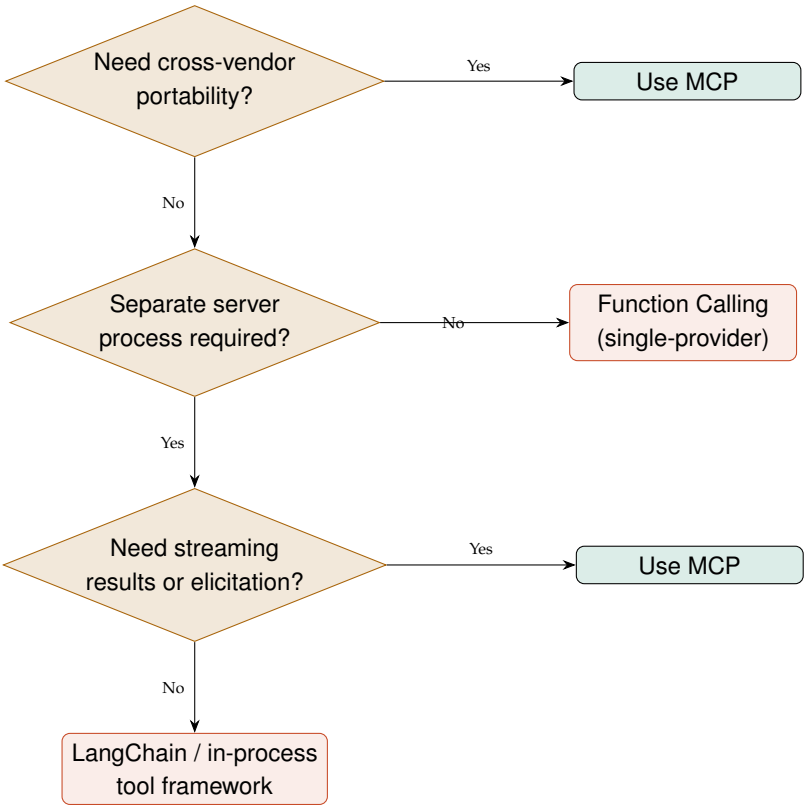

Figure 1.2: Decision flowchart for choosing MCP over alternatives.

Mock Interview Questions

Q1.1 [*Theory*] *What problem does MCP solve that raw function calling does not?*

Answer: MCP solves the $N \times M$ integration problem: before MCP, every combination of model provider and tool required a custom integration. MCP introduces a shared protocol so that any MCP-compliant host (model) can connect to any MCP-compliant server (tool/data) without bespoke glue code. Additionally, MCP adds features absent from provider-specific function calling: cross-vendor portability, separate server processes, native streaming of partial results, elicitation (server-initiated user input), and standardised OAuth 2.1 authorisation.

Q1.2 [*Theory*] *Which RPC specification underpins MCP, and why was it chosen over alternatives like gRPC or GraphQL?*

Answer: MCP uses **JSON-RPC 2.0**. It was chosen because: (1) it is trivially human-readable, easing debugging; (2) it maps naturally to the request/response/notification triad that LLM tool use requires; (3) it is transport-agnostic — the same message format works over `stdio` and HTTP without changes; (4) gRPC requires Protobuf schemas and HTTP/2, which complicates subprocess-spawning use cases; (5) GraphQL is query-centric and poorly suited to RPC semantics.

Q1.3 [*Theory*] *Name the four primitive capability categories MCP defines and give one concrete example of each.*

Answer: (1) **Tools** — executable actions with side effects, e.g. `send_email`; (2) **Resources** — readable data blobs addressed by URI, e.g. `file:///repo/README.md`; (3) **Prompts** — reusable, parameterised prompt templates, e.g. a `code_review` prompt that accepts a diff; (4) **Sampling** — a server-initiated request asking the host to run an LLM completion on its behalf.

Q1.4 [*Syntax*] *Write the minimal valid JSON-RPC 2.0 request to call an MCP tool named `get_weather` with argument `city="Tokyo"`.*

Answer:

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": { "city": "Tokyo" }
  }
}
```

The `id` field must be a string or integer (not null) for requests that expect a response; omit `id` only for JSON-RPC notifications.

Q1.5 [*Syntax*] *What is the correct JSON-RPC error response when a method is not found?*

Answer:

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "error": {
    "code":    -32601,
    "message": "Method not found",
    "data":    { "method": "tools/unknown" }
  }
}
```

Error code `-32601` is the JSON-RPC 2.0 standard "Method not found" code. A result field must *not* be present when an error is present.

Q1.6 [*Theory*] *How does MCP relate to the A2A (Agent-to-Agent) protocol?*

Answer: They are complementary, not competing. MCP is a *vertical* protocol: it connects an agent (host+client) to capabilities (tools, resources). A2A is a *horizontal* protocol: it enables two autonomous agents to delegate tasks to each other at the intent level. A production multi-agent system uses A2A for inter-agent orchestration and MCP for each individual agent's tool access layer.

Q1.7 [*Theory*] *Give two reasons why an enterprise might prefer MCP over embedding tool logic directly in a LangChain agent.*

Answer: (1) **Process isolation**: An MCP server runs in its own process or container, so a crash or security flaw in the tool does not take down the agent process. (2) **Reusability**: The same MCP server can be consumed by Claude Desktop, a Cursor plugin, and a custom Python agent without any code change, because the protocol contract is stable and vendor-neutral.

Q1.8 [*Production*] *A team hardcoded* `"protocolVersion":"2024-11-05"` *in their MCP server. Describe the failure mode and the fix.*

Answer: The server will announce an obsolete version during the `initialize` handshake. A 2025-era client that only advertises `2025-03-26` will either reject the connection outright or fall back to a minimal feature set, silently disabling streaming results and elicitation. The fix is to import `LATEST_PROTOCOL_VERSION` from the SDK and return it dynamically, or to announce a list of supported versions and let the client select via the negotiation algorithm.

Q1.9 [*Production*] *A customer support MCP server exposes an* `order_lookup` *tool. A bug causes it to return orders from other customers. What MCP-level controls should have prevented this?*

Answer: (1) **OAuth scopes**: The access token should carry a customer-specific scope (e.g. `orders:read:cust_123`) that the server validates before executing the query. (2) **Input validation**: The server's tool handler should compare the requested order ID's `customer_id` column against the authenticated identity in the token. (3) **Audit logging**: Every `tools/call` invocation should be logged with the token subject so anomalies are detectable.

Q1.10 [*Production*] *Your MCP server is deployed as a container in Kubernetes. Latency spikes to 10 s on every first request after a pod restart. What is the most likely cause and fix?*

Answer: The most likely cause is a cold-start initialisation that happens on the first `tools/call` rather than at server startup — e.g. lazy database connection pool creation or loading a large ML model. Fix: move all heavy initialisation to the server's startup path (before the `initialize` response is sent), and configure a Kubernetes readiness probe that only passes *after* initialisation completes, preventing traffic from reaching the pod until it is warm.

Q1.11 [*Theory*] *What distinguishes an MCP* notification *from an MCP* request*?*

Answer: A **request** has a non-null `id` field and the recipient *must* send a response (result or error). A **notification** omits the `id` field entirely and the recipient must *not* send a response. MCP uses notifications for fire-and-forget messages such as `notifications/progress` and `notifications/cancelled`. Mixing them up — e.g. sending a response to a notification — violates the JSON-RPC 2.0 spec and can cause the other side to stall waiting for a message that never arrives.

CHAPTER 2

MCP Architecture

Chapter Overview

Every MCP system is easier to reason about once the Host, Client, and Server boundaries are clear. The Host owns user experience and policy, the Client owns one protocol session, and the Server exposes tools, resources, and prompts behind a negotiated capability contract. Treat this chapter as the wire-level map for initialization, routing, and failure isolation.

2.1 Host, Client, Server Model

MCP defines three distinct actors. Their separation of concerns is deliberately strict, and interview questions often probe exactly where each boundary lies.

Host The application that owns the LLM session and presents a UI to the user. Examples: Claude Desktop, Cursor IDE, a custom Python orchestrator. The Host is responsible for (a) instantiating one or more MCP Clients, (b) deciding which tool calls to approve (human-in-the-loop), and (c) injecting resource content into the model context window.

Client A software component *inside* the Host that manages one MCP session to exactly one MCP Server. The client speaks JSON-RPC 2.0, handles the lifecycle (initialize → operate → shutdown), and exposes a typed API to the Host. In the Python SDK, this is the `ClientSession` object.

Server An independent process or network service that exposes capabilities (tools, resources, prompts). It responds to requests from the Client and may send server-initiated notifications. In the Python SDK, `@server.tool()` registers handlers.

Key Design Decisions

One client per server. A Host that connects to five MCP servers must create five independent Client instances, each with its own session state. This prevents capability namespace collisions and simplifies error isolation.

The server has no direct LLM access. A server that needs to invoke the LLM (e.g. to summarise a document) must do so through the `sampling/createMessage` request to the Client, which forwards it to the Host, which decides whether to approve the completion. The server never holds an API key.

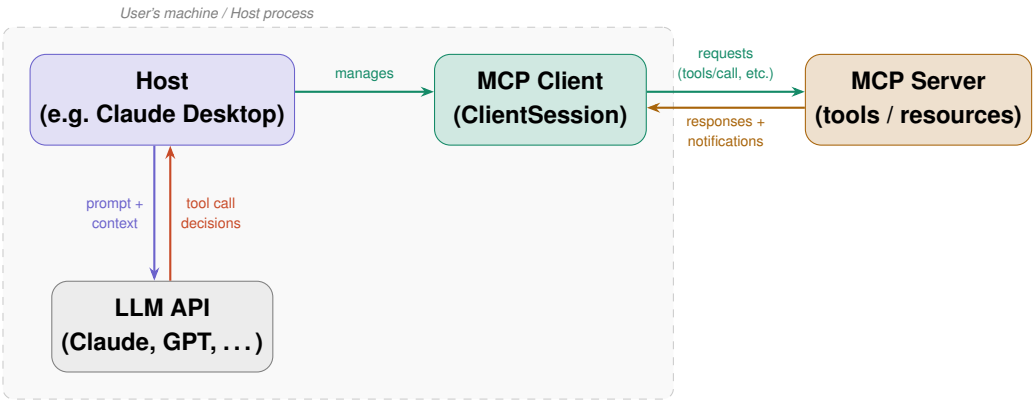

Figure 2.1: MCP three-actor model. The Host owns the LLM session; the Client manages the MCP session; the Server exposes capabilities.

2.2 JSON-RPC 2.0 Message Mechanics

Message Types

MCP uses exactly three JSON-RPC 2.0 message types:

1. **Request** — has `id` (integer or string), expects a Response.
2. **Response** — has matching `id`, contains either `result` or `error` (never both).
3. **Notification** — no `id`, no response expected or sent.

```
// Request
{ "jsonrpc":"2.0", "id":1, "method":"tools/list", "params":{} }

// Success response
{ "jsonrpc":"2.0", "id":1, "result":{ "tools":[...] } }

// Error response
{ "jsonrpc":"2.0", "id":1, "error":{ "code":-32601,
    "message":"Method not found" } }

// Notification (no id -- no response expected)
{ "jsonrpc":"2.0", "method":"notifications/progress",
  "params":{ "progressToken":"tok_1", "progress":50, "total":100 } }
```

Listing 2.1: The three JSON-RPC 2.0 message shapes

Error Codes

Message ID Rules

IDs must be integers or strings; `null` IDs are forbidden for requests (they are used internally by JSON-RPC 2.0 to mark notifications and would cause a protocol violation). IDs must be unique within a session; reuse causes response routing failures in multiplexed sessions.

Batching

JSON-RPC 2.0 technically supports request batching (an array of requests). The current MCP specification *explicitly discourages* batching — each request must be sent individually. Hosts

Code	Meaning	MCP Usage
−32700	Parse error	Malformed JSON received
−32600	Invalid Request	Missing `jsonrpc` or `method` field
−32601	Method not found	Unknown MCP method
−32602	Invalid params	Schema validation failure on `params`
−32603	Internal error	Unhandled exception in handler
−32000	MCP: Resource not found	URI resolves to nothing
−32001	MCP: Tool execution error	Handler returned `isError:true`

Table 2.1: JSON-RPC 2.0 standard error codes and MCP-specific extensions.

that implement batching for performance should note that the spec may enforce this restriction in future versions.

2.3 Capability Negotiation

Every MCP session begins with an `initialize` request/response pair that declares and discovers capabilities before any tool calls are made. This handshake is the most-tested part of the MCP lifecycle.

The Handshake

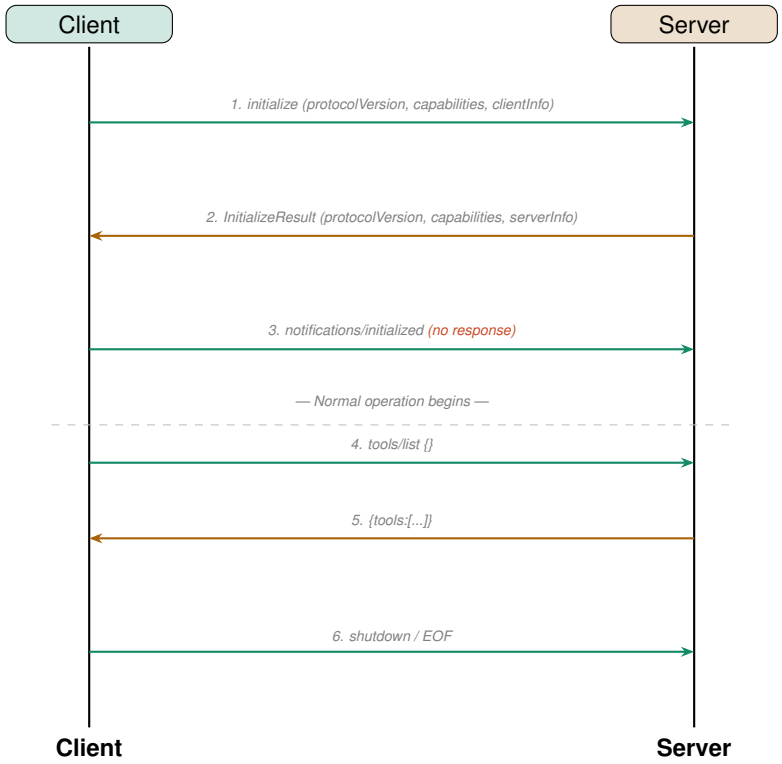

Figure 2.2: MCP session initialization handshake and first tool list.

Capability Fields

The `initialize` request from the client includes: